Game Engine — Design & Scaling Doc

“Hit New Game and Go.” How the system works today, and the roadmap to scale it from one game to many.

Engine repo: `agadabanka/game-engine` · Hub: `hub-production-6d28.up.railway.app` · 2026-06-14

1. Overview

The game engine turns a **one-line concept** (genre + theme + any wishes) into a complete, polished, single-player game — scaffolded, designed, quality-gated, art- and music-skinned, deployed, recorded to video, and registered on a central hub. The owner supplies only the intent; the `make-game` skill carries every standing preference and runs the whole pipeline, ending with links.

The defining property is **inheritance**: every new game is a thin skin over a deep, shared stack, so each game starts where the last left off. Anything reusable discovered while building one game is promoted back into the engine, so every *future* game gets it for free.

2. Architecture at a glance

Figure 1 — The platform stack (left), the make-game pipeline (right), and the hub + shorts subsystem (bottom). The owner’s one-line concept enters top-left; a shipped game exits top-right.

3. Platform architecture — the tiers every game inherits

Tier	What every game gets

5 · The game (the skin)	The only per-game work: hero/roster, 5 themed worlds, level data, art, music, system prompt.
4 · Studio SDK `engine/sdk/studio.js`	**Studio.Toon** (procedural character rigs, 15+ animation states) · **Studio.Brawl** (brawler + CPU policies + felt-fun scorer) · **Studio.Juice** (particles, rings, sparks, shake) · **Studio.RNG** (seeded determinism) · **Studio.Audio** (procedural SFX + music) · the deterministic **harness** (`window.__game` / `window.__rec`) that powers eval and recording. Kept byte-identical across the template and each game's vendored copy.
3 · Phaser 4 engine	Scenes, materials, the in-game level builder, merge.
2 · Platform services	Express **server** + volume-backed **store** · **Gemini** image pipeline (art) · **Lyria** music pipeline · the **evaluation suite** (0-death gate, felt-fun model, autopilot recorder, vision judge) · the **notes** → **diary** → **issues** loop.
1 · Deploy	Railway — one project per game, health-checked, exposing `/health`, `/api/meta`, `/api/diary`, `/api/notes`.

Above every game sits the **hub** — a meta-layer that doesn't run a game but *observes* them all: per-game cards, a combined diary timeline, an aggregated notes-triage feed, and the shorts feed (§5).

4. The make-game pipeline

When the owner gives a concept, these stages run in order; each must land (`GAME_META.json` tracks status).

1. **Trigger / skill load.** The concept is the only input. The `make-game` skill loads with the owner's standing preferences (polished toony look, lots of juice, variety as a theme, 5 levels, single-player vs CPU). No follow-up questions.
2. **Scaffold.** `scripts/new-game.mjs` clones the base (`engine/game-template`), rebrands it, writes `GAME_META.json`, creates & pushes the GitHub repo, registers it on the hub.
3. **Identity.** Name, tagline, hero/roster, worlds list → `GAME_META.json`.
4. **Levels.** 5 themed levels as data in `src/game/levels.js` — each a distinct biome.
5. **Gate (the eval).** Extend `eval.mjs`, then run until green: a **menu smoke-test**, the autopilot **wins every level**, two identical runs are **byte-deterministic**, the run scores **FUN ≥ 70** (comebacks beat sweeps), non-black on **both renderers**. RNG genres seed-scan in headless (~70× faster) and bake winning seeds.
6. **Feel.** Animation states on every actor, hitstop/shake/flash, HUD, particles, procedural SFX.
7. **Art.** Gemini backdrops per world + title keyart (`tools/art.mjs`); bottom third gameplay-clean.
8. **Music.** One Lyria loop per world + a title theme (`tools/music.mjs`).
9. **Deploy.** Railway, one project per game; verify live render non-black.
10. **Videos.** `tools/record.mjs` renders each level's run to MP4 + music; montage; YouTube + playlist; stills.
11. **Shorts.** `make-shorts <id>` records levels 1/3/5 (mobile-encoded ~2 MB, honours `?level`, dismisses menus, muxes music); `host-shorts <id>` uploads + auto-wires the registry; deploy the hub. Zero shorts config for a new game.
12. **Loop closed.** Register in `hub/games.json`; push the game repo + the engine branch; reply with repo · URL · playlist · diary · shorts.

Runs throughout: the **diary** (`DIARY.md`) is the deliverable the owner reads; and **evolve the engine** — anything reusable is promoted into `engine/sdk/studio.js` so the next game starts stronger.

5. The hub & the shorts subsystem

A vertical, TikTok-style gameplay feed. Four engine-level parts; a new game inherits all of it for free:

Part	Responsibility & baked-in fixes
Recorder `make-shorts.mjs`	Films distinct levels off the live deploy. Honours `?level` (no `reset()`), `gotoLevel(i)` dismisses a menu, `menuStart` taps space for “press-space” shells, muxes per-level music, **encodes mobile** (720×1280 / CRF 28 ≈ 2 MB). **Auto-plan**: defaults to levels 1/3/5; override via a `meta.shorts` block.
Host `host-shorts.mjs`	(Re)creates the `shorts` Release, uploads mp4s, **auto-wires** the asset URLs into `hub/games.json`.
Proxy `hub/server.js /v`	Streams private-repo assets with auth + range forwarding, same-origin so `<video>` seeks work.
Player `shorts.html`	**Audio**: muted-then-unmute + re-assert unmute on every gesture (iOS rule). **Buffering**: tight 4-element decoder window + self-healing watchdog + spinner-clears-on-any-signal. **Variety**: round-robin interleave.

Caveats: `?level=N` is the uniform level-jump contract — a level-select shell must gate its title on `mode!=='play'`, expose `gotoLevel`, and unlock all cards. Credentials (`GH_TOKEN`, `GEMINI_SA_JSON`, `RAILWAY_TOKEN`, `YT_*`) gate stages and degrade gracefully. Determinism (`Studio.RNG` + fixed frames) is the backbone.

6. Where the system is today (the honest baseline)

Dimension	Today	Implication for scale
Production	~1 polished game per focused session, pipeline executed stage-by-stage with a human/agent driving.	Throughput is the first ceiling — needs one-button + parallel runs.
Source of truth	Split: some stages are engine *code*, some are *skill prose* I interpret.	Prose doesn't scale or self-test; migrate to code.
Hosting	One Railway project per game; shorts on GitHub Releases via an auth proxy.	Per-game ops + cost grows linearly; proxy is a hop.
Registry	A `games.json` seed + volume-backed store.	Fine for ~dozens; not for hundreds + search.
Quality	Per-game eval gate, run at build time.	A shared-SDK change can silently break many games.
Feedback	The autopilot + felt-fun model judge fun; no real-player signal.	Design can't learn from actual players yet.

7. Next steps — scaling the system

Five levers, each with concrete next steps. The goal: go from "one carefully-built game" to "many games, produced fast, quality holding, cost bounded."

7.1 Automation & throughput — close the loop to one button

- **An autonomous `make-game` runner** that executes all 12 stages headlessly with checkpoints, auto-retry, and a status report — so "make a game" is one command, not a guided session.
- **Parallel production**: run N builds concurrently (git worktrees + one Railway project each) behind a queue/orchestrator. Determinism makes parallel runs safe and reproducible.
- **Batch planner**: turn a list of concepts ("make 10 games across these genres") into parallel pipelines with a shared dashboard.

7.2 Skill → programmatic migration — harden prose into code

- Move what still lives as *skill instructions* into deterministic **engine tools with typed contracts + tests**, so the engine is the source of truth and a stage can run without a human in the loop.
- Priorities: the diary as a **generated artifact** from structured build events (not hand-written); art/music as **parameterized, cached** tools; the level-design heuristics as a reusable **level-kit** module; the make-game stage-runner itself as code.
- Outcome each migration buys: fewer regressions, self-documenting behavior, and the ability to run unattended.

7.3 Infrastructure at scale — hosting, registry, assets

- **Multi-tenant hosting**: games are mostly client-side Phaser — serve many from one service (or static bundles on a CDN) instead of a Railway project each. Cuts ops + cost from O(N) toward O(1).
- **Registry as a database**: replace `games.json` + volume with Postgres + an admin API (search, categories, featured). The hub becomes a real storefront backend.
- **Shorts/video on a CDN**: move clips to object storage + CDN (e.g. R2/S3) with signed URLs; retire the per-request proxy hop. The player stays unchanged.
- **Cost & quota governance**: budget caps + a usage dashboard for Gemini / Lyria / YouTube / Railway; cache generated assets so identical art/music is never re-billed.

7.4 Quality at scale — keep the bar as N grows

- **Cross-game eval in CI**: on every engine/SDK change, run every game's gate (the SDK is shared — one change can break many). Block the merge on red.
- **Automated visual QA**: the vision judge + screenshot diffing across all games to catch black screens, broken menus, off-theme art — the exact bug classes hit while building the shorts feed.
- **A "golden games" canary set** the engine must always pass before any release.

7.5 Distribution, feedback & monetization

- **The hub as a public storefront**: categories, search, featured rows, with the mobile shorts feed as the discovery surface (already mobile-first).
- **Analytics + a player-feedback loop**: instrument plays (level reach, deaths, session length) and feed real signals back into the **felt-fun model**, so level design learns from actual players, not just the autopilot.
- **Accounts, saves, leaderboards** → then **monetization** (ads on shorts, cosmetic IAP, or a catalog subscription).

7.6 Suggested sequencing

Do the safety net (2) before pushing the throughput pedal hard — scaling count without cross-game CI multiplies breakage.